LARAVEL LAB ASSIGNMENT

Book Registration and Viewing — Complete Solution

Student: Murtaza Arshad Rahimi **Father's name:** Kalimullah Arshad Rahimi

This solution follows the provided lab sheet: create a books table, register books with validation, and show each authenticated user only their own books.

1. Create the Model and Migration

Run this command in the terminal from your existing Laravel project:

```
php artisan make:model Book -m
```

Open the newly created migration in database/migrations and define the table:

```
public function up(): void
{
    Schema::create('books', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->cascadeOnDelete();
        $table->string('title', 150);
        $table->string('author', 100);
        $table->unsignedInteger('pages');
        $table->timestamps();
    });
}
```

Ensure the migration includes these imports:

```
use Illuminate\\Database\\Migrations\\Migration;
use Illuminate\\Database\\Schema\\Blueprint;
use Illuminate\\Support\\Facades\\Schema;
```

Run the migration:

```
php artisan migrate
```

Configure .env with your XAMPP MySQL database settings. Example only:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

Replace DB_DATABASE with the actual database name used by your project.

2. Configure the Book Model

File: app/Models/Book.php

```
<?php

namespace App\\Models;

use Illuminate\\Database\\Eloquent\\Factories\\HasFactory;
use Illuminate\\Database\\Eloquent\\Model;

class Book extends Model
{
    use HasFactory;

    protected $fillable = [
        'title',
        'author',
        'pages',
        'user_id',
    ];
}
```

3. Create the Controller

Run:

```
php artisan make:controller BookController
```

File: app/Http/Controllers/BookController.php

```
<?php

namespace App\\Http\\Controllers;

use App\\Models\\Book;
use Illuminate\\Http\\Request;

class BookController extends Controller
{
    public function index()
    {
        $books = Book::where('user_id', auth()->id())
            ->latest()
            ->get();

        return view('dashboard', compact('books'));
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|string|max:150',
            'author' => 'required|string|max:100',
            'pages' => 'required|integer|min:1|max:10000',
        ]);

        $validated['user_id'] = auth()->id();

        Book::create($validated);

        return redirect()
            ->route('dashboard')
            ->with('success', 'Book registered successfully!');
    }
}
```

4. Configure the Routes

File: routes/web.php. Preserve existing authentication routes. Replace the existing dashboard route instead of creating a duplicate.

```
use App\\Http\\Controllers\\BookController;

Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [BookController::class, 'index'])
        ->name('dashboard');

    Route::post('/books', [BookController::class, 'store'])
        ->name('books.store');
});
```

If the file already has PHP opening tags or imports, add only the missing import and routes; do not duplicate them. Keep your existing login and logout routes.

5. Create the Dashboard View

File: resources/views/dashboard.blade.php. The following standalone view uses Bootstrap 5.3.3. If your existing dashboard has a layout or navigation, integrate the form and table into it instead of removing needed authentication/layout code.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Book Registration Dashboard</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
```

```
          rel="stylesheet">
</head>
<body class="bg-light">
    <nav class="navbar navbar-dark bg-dark">
        <div class="container">
            <a class="navbar-brand" href="{{ route('dashboard') }}">My Reading List</a>
            <div class="d-flex align-items-center gap-3">
                <span class="text-white">Welcome, {{ auth()->user()->name }}</span>
                <form action="{{ route('logout') }}" method="POST">
                    @csrf
                    <button type="submit" class="btn btn-outline-light btn-sm">Logout</button>
                </form>
            </div>
        </div>
    </nav>

    <div class="container py-5">
        <h1 class="fw-bold">Book Registration Dashboard</h1>
        <p class="text-muted">Register books and manage your personal reading list.</p>

        @if (session('success'))
            <div class="alert alert-success alert-dismissible fade show">
                {{ session('success') }}
                <button type="button" class="btn-close" data-bs-dismiss="alert"
                        aria-label="Close"></button>
            </div>
        @endif

        <div class="card shadow-sm mb-5">
            <div class="card-header bg-primary text-white">
                <h5 class="mb-0">Register a New Book</h5>
            </div>
            <div class="card-body">
                <form action="{{ route('books.store') }}" method="POST">
                    @csrf
                    <div class="mb-3">
                        <label for="title" class="form-label">Book Title</label>
                        <input type="text" name="title" id="title" maxlength="150"
                               value="{{ old('title') }}"
                               class="form-control @error('title') is-invalid @enderror"
                               required>
                        @error('title')
                            <div class="invalid-feedback">{{ $message }}</div>
                        @enderror
                    </div>

                    <div class="mb-3">
                        <label for="author" class="form-label">Author</label>
                        <input type="text" name="author" id="author" maxlength="100"
                               value="{{ old('author') }}"
                               class="form-control @error('author') is-invalid @enderror"
                               required>
                        @error('author')
                            <div class="invalid-feedback">{{ $message }}</div>
                        @enderror
                    </div>

                    <div class="mb-3">
                        <label for="pages" class="form-label">Number of Pages</label>
                        <input type="number" name="pages" id="pages" min="1" max="10000"
                               value="{{ old('pages') }}"
                               class="form-control @error('pages') is-invalid @enderror"
                               required>
                        @error('pages')
                            <div class="invalid-feedback">{{ $message }}</div>
                        @enderror
                    </div>

                    <button type="submit" class="btn btn-primary">Register Book</button>
                </form>
            </div>
        </div>

        <div class="card shadow-sm">
            <div class="card-header bg-dark text-white">
                <h5 class="mb-0">My Books</h5>
            </div>
            <div class="card-body">
                @if ($books->isEmpty())
                    <div class="alert alert-info mb-0">
```

```
                    You have not registered any books yet.
                </div>
            @else
                <div class="table-responsive">
                    <table class="table table-bordered table-hover align-middle">
                        <thead class="table-primary">
                            <tr>
                                <th>No.</th>
                                <th>Title</th>
                                <th>Author</th>
                                <th>Pages</th>
                            </tr>
                        </thead>
                        <tbody>
                            @foreach ($books as $book)
                                <tr>
                                    <td>{{ $loop->iteration }}</td>
                                    <td>{{ $book->title }}</td>
                                    <td>{{ $book->author }}</td>
                                    <td>{{ $book->pages }}</td>
                                </tr>
                            @endforeach
                        </tbody>
                    </table>
                </div>
                <p class="text-muted mb-0">Total books: {{ $books->count() }}</p>
            @endif
        </div>
    </div>
</div>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

6. Run and Test

```
php artisan optimize:clear
php artisan route:list
php artisan serve
```

Open the local URL shown by Laravel (usually http://127.0.0.1:8000), log in, and visit the dashboard.

7. Required Test Data

Title	Author	Pages
Learning PHP	Sara Ahmad	120
Laravel Basics	Ali Hassan	180

The table should show the newest book first. Refresh the page to verify the records remain saved.

8. Final Checklist

- The books table exists in MySQL.
- Both required books can be registered.
- Empty or invalid values are rejected.
- Old title and author values remain after a validation error.
- Books remain after refreshing the dashboard.
- A second user cannot see the first user's books.
- Logging out prevents access to the authenticated dashboard.
- The form and table display correctly using Bootstrap.

9. Common Problems

Books table does not exist: run `php artisan migrate` and check the database settings.

Dashboard route not defined: check that the GET /dashboard route is named dashboard.

Dashboard view not found: verify the file is resources/views/dashboard.blade.php.

user_id fillable error: include user_id in the Book model's $fillable array.

Logout route not defined: keep and use the logout route supplied by your existing authentication system.

Unknown user_id column: check the migration and run it against the correct database.

Submission note: This is the code solution and implementation guide. Run it in your existing Laravel project and verify the checklist before demonstrating it to your instructor.